Product Requirements Document (PRD): Pockett Core Platform MVP

Document purpose: This PRD defines the product scope, features, and requirements for the Pockett platform—a client-centric document delivery and collaboration workspace for professional services. It is intended as a client and stakeholder deliverable for alignment on scope, behaviour, and acceptance criteria.

Audience: Product owners, stakeholders, and implementation teams.

---

1. Overview

Pockett is a professional client portal that connects organisations' existing Google Drive to a structured workspace. Users organise work by clients and projects, control access via roles and personas, and deliver documents through a branded experience—without migrating files or changing where data lives. This document describes the implemented and planned features for the Core Platform MVP and the billing model that supports tiered plans (Standard, Pro, Business, Enterprise).

2. Onboarding Flow

Purpose: Ensure every user has a dedicated workspace immediately upon signup.

- [x] **Trigger**: New user signup via Auth provider (Google).
- [x] **Routing**:
  - [x] If User has *NO* organization memberships -> Redirect to `/onboarding`.
  - [x] If User has existing memberships -> Redirect to `/d` (organizations list page).
- [x] **Feature: "Name your Workspace"**:
  - [x] Simple form input for Organization Name.
  - [x] Auto-generates a URL-friendly slug.
  - [x] **Action**: Provisions a new Organization (Tier: Free) and assigns the user as `ORG_OWNER`.

3. Organization Level

Purpose: The root container for all business data, settings, and team access.

- [x] **Structure**:
  - [x] **Multi-Tenancy**: Users can belong to multiple organizations (switching via User Profile dropdown, though currently scoped to single active view).
  - [x] **Role-Based Access Control (RBAC)**:
    - [x] `ORG_OWNER`: Full administrative access (Billing, Settings, Client Creation).
    - [x] `ORG_MEMBER`: Standard access (View/Edit Projects).
    - [x] `ORG_GUEST`: Restricted access (Project-scoped).
- [x] **Organizations List (`/d`)**:
  - [x] Shows all organizations the user belongs to in grid/list view.
  - [x] Default organization highlighted.
  - [x] "Create Organization" button for creating additional workspaces.
  - [x] Clicking an organization navigates to `/o/[slug]` (clients list).
- [x] **Dashboard**:
  - [x] Displays global navigation (App Sidebar: Projects, Members, Shares, Insights, Sources, Connectors).
- [x] **Connectors**: Org-level cloud storage connections at `/o/[slug]/connectors`. Schema supports multiple connector types (Google Drive today; Dropbox/OneDrive planned); each connector has `type` and

`externalAccountId` (unique per org+type). Used for project folder sync and Import from Drive in file browser.
RLS: `portal.connectors` has organization-level RLS so users only see connectors for their organizations.

- ☑ **Folder Setup**: When Google Drive connector is initialized, creates `.pockett` root folder and organization folder with strict permission restrictions (owner-only, no inheritance from parent). See [File Management Security] for details.

4. Client Management

Purpose: Organise work by the customer or entity being served.

- ☑ **Data Model**: Clients are children of an Organization.
- ☑ **UI Components**:
  - ☑ **Sidebar Selector**: Dropdown to switch between active Clients.
  - ☑ **Persistence**: Remembers the last selected Client per Organization (LocalStorage).
  - ☑ **Empty State**: Prompts to create a Client if none exist.
- ☑ **Feature: Create Client**:
  - ☑ **Access**: `ORG_OWNER` only.
  - ☑ **Input**: Client Name, Industry/Sector.
  - ☑ **Output**: Creates a Client entity in DB.

5. Project Management

Purpose: The primary unit of work (engagement, case, or audit).

- ☑ **Data Model**: Projects belong to a Client and are linked 1:1 with a connector root folder (e.g. Google Drive); the folder ID is stored in `connectorRootFolderId`.
- ☑ **UI Components**:
  - ☑ **Project List**: View all projects for the active Client.
    - ☑ Toggle: Grid View / List View.
  - ☑ **Sidebar Navigation**:
    - ☑ "Projects" item expands to show active Project Tabs when inside a project.
- ☑ **Feature: Create Project**:
  - ☑ **Input**: Project Name, Start Date, Description.
  - ☑ **Integration**: Automatically creates a structured Google Drive Folder (`[Client Name] / [Project Name]`).
  - ☑ **Sync**: Stores the connector root folder ID in DB (`connectorRootFolderId`) for direct access.
- ☑ **Project Creator as Project Lead**: When a project is created, the creator is automatically added as a Project Lead member. This ensures the org owner (or any user who creates the project) sees the file list and has full project access without special-case logic; file visibility remains strictly persona-based.
- ☑ **Project Settings (Project Lead only)**:
  - ☑ **Settings icon**: Prominent settings icon beside the project name (large bold title); visible only to users with the Project Lead persona for that project.
  - ☑ **Role access config**: In-code JSON config (`PROJECT_ROLE_ACCESS.canViewProjectSettings: ['Project Lead']`) determines who can view and use project settings; intended to move later to permissions or `project_personas` table.
  - ☑ **Settings modal**:
    - **Project Properties**: Name, Description, Save. When the project is closed (`isClosed: true`), property fields and Save are disabled; user must reopen to edit.

        - **Close project** (amber): Sets `isClosed: true`. Project becomes view-only for current members. All project members who are org guests (`ORG_GUEST`) are removed and their Google Drive folder access is revoked.
        - **Reopen project** (amber): Shown when `isClosed: true`. Sets `isClosed: false`; only Project Lead can reopen.
        - **Delete project** (red): Soft delete. Sets `isDeleted: true`; removes all project members; revokes all Google Drive permissions on the project folder (restrict to owner only). The project folder is **not** deleted in Google Drive so the org admin can still access files natively outside Pockett. No other DB records or Drive files are deleted.

- ☑ **Project lifecycle flags**: `Project.isClosed` (Boolean, default false) and `Project.isDeleted` (Boolean, default false). Deleted projects are excluded from hierarchy and all project fetches; they are hidden from everyone in the portal.
- ☑ **Project Workspace (Tabs)**: Tab and sidebar visibility are restricted by persona. See **§7.6 Permission-based UI: Who can see what** for the full matrix.
    1. ☑ **Files**: Document management (default view); full file browser, uploads, Import from Drive, folder upload. Visible to all personas with project access.
    2. ☑ **Members**: Member list, invitations, personas (see §7). Visible to Team Member, Project Lead, Client Owner, Org Owner only.
    3. ☐ **Shares**: User/guest sharing settings (placeholder). Same visibility as Members.
    4. ☑ **Insights**: Project-level insights dashboard (recent/trending/storage/sharing views). Same visibility as Members.
    5. ☐ **Sources**: Data sources & connectors (placeholder; org-level Connectors at `/o/[slug]/connectors` for Google Drive). Same visibility as Members.
    6. ☑ **Settings**: Project properties, close/reopen, delete. Visible to Project Lead, Client Owner, Org Owner only; implemented as a tab after Sources.

6. File Management

Purpose: Secure, robust, and familiar document handling powered by Google Drive.

- ☑ **Architecture**: "Headless" Drive integration. Pockett acts as the UI, Google Drive acts as the storage/backend.

- ☑ **Security**:

    - ☑ **Direct-to-Drive Uploads**: Files are streamed directly from Browser to Google (TLS 1.3), bypassing Pockett servers (Resumable Upload Protocol).
    - ☑ **Scoped Access**: System uses a Service Account or OAuth Scope limited to specific folders.
    - ☑ **Folder Permission Restrictions**:
        - ☑ **Centralized Access Control**: All folder permissions are managed exclusively through Pockett Portal project membership. Google Drive's native sharing is disabled to prevent unauthorized access.
        - ☑ `.pockett` **Root Folder**: When created during organization setup, permissions are restricted to owner-only with no inheritance from parent folder. All non-owner permissions are removed to ensure strict access control.
        - ☑ **Organization Folders**: Each organization folder created under `.pockett` is restricted to owner-only access with no inheritance from `.pockett` parent. This ensures complete isolation of organization data.
        - ☑ **Project-Level Permissions**: Access to project folders is granted automatically when users join projects via invitation (Project Lead and Team Member personas receive `can_edit` access). Permissions are revoked automatically when members are removed from projects.
        - ☑ **Enforcement**: Applied automatically during `setupOrgFolder()` when Google Drive connector is initialized. Both new and existing folders are secured to prevent unauthorized

access. This ensures that all access is controlled through Pockett Portal's project membership system, not through Google Drive's native sharing mechanisms.

- [x] **Feature: File Browser**:
  - [x] **Visuals**: Clean, table-based layout (Name, Owner, Date modified, File size). Column headers in Title case; Sort column right-aligned with row action menu.
  - [x] **Icons**: Dynamic file-type icons (PDF, Sheets, Docs, Images, etc.).
  - [x] **Navigation**: Breadcrumbs for folder traversal.
  - [x] **Loading state**: Toolbar (Add, Type, People, Modified, Source, Search) disabled while the file list is loading.
  - [x] **Filters**: Type (multi-select with checkboxes; "All types" indeterminate when some selected; stays open until Done), People (Any / Owned by me / Not owned by me), Modified (Any / 7d / 30d / 1y), Source (filter by connector). All filter dropdowns use `text-xs` and header + Done button.
  - [x] **Sort**: Dedicated Sort column (icon + "Sort" label) with dropdown: Sort by (Name, Date modified, Date modified by me, Date opened by me), Sort direction (A–Z, Z–A), Folders (On top, Mixed with files). Default: Name, A–Z, Folders on top.
  - [x] **Refresh**: Button next to search to refresh list (e.g. after renaming in Google Docs).
  - [x] **Actions**: Row action menu (Preview, Edit in Google Docs, Open Folder in Drive, Download, Share, Version history, Bookmark, Set Due Date, Rename/Copy/Move/Delete when callbacks provided). Menu items `text-xs`; menu stays open until user dismisses.
  - [x] **Long names**: Truncated file names show full name in tooltip.
  - [x] **Direct-to-Drive**: Add menu indicates uploads go directly to Google (never through Pockett servers).
- [x] **Feature: Add Menu**:
  - [x] **New folder** (and New Doc/Sheet/Slide/etc. via submenu).
  - [x] **From your computer**: Upload files; Upload folder (retains folder structure in Drive; in-app confirmation modal).
  - [x] **Import from Google Drive**: Opens Google Picker with **My Drive** and **Shared Drives** tabs; starts at root (top-level folders/files), LIST view; user traverses hierarchy and multi-selects; selection flows to import confirmation dialog (copy/shortcut).
- [x] **Feature: Upload Experience**:
  - [x] **UX**: Bottom-right floating progress modal (similar to Gmail/Drive).
  - [x] **Capabilities**:
    - [x] **Multi-file Batching**: Queue multiple files.
    - [x] **Background Processing**: Continue navigating while uploading.
    - [x] **Conflict Resolution**: Detects duplicates. User can choose to **Rewrite** (Replace) or **Keep Both** (Auto-rename).
    - [x] **Smart Tracking**: If a file is renamed (e.g. `Report.pdf` -> `Report_1.pdf`) due to conflict, the UI captures the *final* name.
  - [x] **Post-Upload**: "Show file location" button highlights the new file in the list.
- [ ] **Feature: File Assignment to External Members** (High Priority):
  - [ ] **Purpose**: Enable Project Leads to assign individual files/folders from the `general` folder to External Collaborator and Client Contact personas. This provides granular file-level sharing beyond project-level access.
  - [ ] **Access Control**: Only Project Leads can assign files to external members. External Collaborator and Client Contact personas do not automatically receive Drive access upon project invitation; they only get access to files explicitly assigned to them.

- [ ] **Assignment UI**:
  - [ ] When Project Lead selects a file/folder in the file browser, show "Assign to Members" option in the row action menu.
  - [ ] Assignment modal lists External Collaborator and Client Contact members for the project.
  - [ ] Allow multi-select assignment (assign one file to multiple external members).
  - [ ] Show visual indicators (badge/icon) in file browser for files that have been assigned to external members.
- [ ] **Google Drive Permission Grant**:
  - [ ] When a file is assigned to an External Collaborator, automatically grant `writer` (can_edit) permission to that specific file/folder in Google Drive.
  - [ ] When a file is assigned to a Client Contact, automatically grant `reader` (can_view) permission to that specific file/folder in Google Drive.
  - [ ] Permissions are scoped to the assigned file/folder only, not the entire project folder.
- [ ] **Assignment Tracking**:
  - [ ] Track file assignments in database (new `FileAssignment` table or extend existing models) to record which files are assigned to which external members.
  - [ ] Store: `fileId` (Google Drive file ID), `projectId`, `memberId` (ProjectMember ID), `assignedAt`, `assignedBy` (userId).
- [ ] **Revoke Assignment**:
  - [ ] Allow Project Lead to revoke file assignments, which removes Google Drive permissions for that file/folder.
  - [ ] Revocation removes the assignment record and revokes the Drive permission.
- [ ] **Scope Limitations**:
  - [ ] File assignment only applies to files/folders within the `general` folder (not `confidential` folder).
  - [ ] Cannot assign files to Project Lead or Team Member personas (they already have full access).
  - [ ] Assignment is file-level, not folder-level inheritance (assigning a folder does not automatically assign all files within it).

- [ ] **Feature: Document Review & Collaboration** (High Priority):
  - [ ] **Review System**: Add comments/feedback functionality with threaded discussions to maintain conversation context.
  - [ ] **Approve/Finalize/Publish Workflow**: Allow guests (External Collaborator, Client Contact) to approve, finalize, or publish documents.
  - [ ] **Publish/Finalize to Lock & Version**: When a document is published/finalized, lock it and create a version snapshot.
  - [ ] **Export to PDF**: Enable export of finalized documents to PDF format.
  - [ ] **Watermark Branding**: Add watermarking with organization branding to exported PDFs.
  - [ ] **"Track" Tab**: Add a "Track" tab beside files to show review status, comments, approvals, and version history.
- [ ] **Feature: Project Templates & Duplication** (High Priority):
  - [ ] **Template Projects**: Define template projects with pre-defined folder structures and template documents. Provide ready-made templates for targeted Lines of Business (LOBs).
  - [ ] **Template Selection**: Allow users to choose a template project when starting a new project to begin with reusable assets.

- ☐ **Duplicate Project**: Enable duplication of existing projects with all folder structures, documents, and configurations.

- ☐ **Feature: Document Relationships & Dependencies** (High Priority):
  - ☐ **Related/Dependent Documents**: Support commitment-based or linked access between documents, not just folder-level access.
  - ☐ **Relationship Management**: Add relationship tracking amongst folders or files (e.g., parent-child, dependencies, references).
  - ☐ **Relationship Tree View**: Display relationship tree visualization showing document dependencies and connections (project task-like structure).

- ☐ **Feature: Client Communication & Follow-ups** (High Priority):
  - ☐ **Automated Consolidated Follow-ups**: Send automated consolidated client follow-up emails on all pending documents.
  - ☐ **Custom Follow-up Messages**: Allow customization of follow-up message templates and scheduling.
  - ☐ **Calendar Integration**: Block calendar through Calendly (or similar) for document discussion scheduling.
  - ☐ **Bi-directional Calendar Requests**: Enable both Team → Client and Client → Team calendar request flows for document discussions.

- ☐ **Feature: UI Enhancements** (High Priority):
  - ☐ **Project Card Images**: Add random/featured images to Project cards for visual appeal and better project identification.

7. RBAC & Permission System

Purpose: Implement comprehensive Role-Based Access Control (RBAC) with hierarchical permissions and in-memory caching for optimal performance.

7.1 RBAC Schema

- ☑ **Schema Structure**: Five-table RBAC model implemented:
  - ☑ `rbac.roles` : Abstract roles (`sys_manager` , `org_member` , `org_guest`)
  - ☑ `rbac.permission_scopes` : Scopes (`organization` , `client` , `project` , `document`)
  - ☑ `rbac.privileges` : Privileges (`can_view` , `can_edit` , `can_comment` , `can_manage`)
  - ☑ `rbac.personas` : Global personas (`sys_admin` , `org_admin` , `proj_admin` , `proj_member` , `proj_guest` , etc.)
  - ☑ `rbac.grants` : Persona + Scope + Privilege mappings
- ☑ **Portal Schema Integration**:
  - ☑ `portal.organization_members` : Links users to organizations with `organizationPersonaId` FK
  - ☑ `portal.project_members` : Links users to projects with `rbacPersonaId` FK
  - ☑ Removed legacy `portal.roles` and `portal.role_permissions` tables

7.2 Permission Computation & Caching

- ☑ **In-Memory Caching**: Permissions computed once on login, cached for 30 minutes
- ☑ **Hierarchical Structure**: Organization → Client → Project permissions stored as nested arrays
- ☑ **Zero DB Queries on Navigation**: All permission checks use cached data (~0-5ms lookup)
- ☑ **Cache Invalidation**: Automatic invalidation on permission changes (member add/remove, invitation acceptance, project deletion)

- ☑ **Performance**: First call ~100-500ms (includes DB query), cached calls ~0-5ms

7.3 Permission Retrieval Flow

1. **Login**: User signs in → `buildUserSettingsPlus()` server action called → Single DB query fetches all permissions → Cached in-memory
2. **Navigation**: All routes check permissions from cache (no DB queries)
3. **UI Gating**: Components receive permissions as props, gate actions based on `canEdit`, `canManage`, `canView`
4. **Permission Changes**: Cache automatically invalidated, rebuilt on next request

7.4 UI Feature Flagging

- ☑ **Upload/Create Actions**: Gated with `canEdit` permission
- ☑ **Settings Button**: Gated with `canManage` permission
- ☑ **Member Actions**: Invite, remove, change role gated with `canManage`
- ☑ **Client Creation**: Gated with `client` scope `can_manage` permission
- ☑ **Organization Selector**: Shows all organizations user belongs to, defaults to `isDefault` organization

7.5 Deployment Version Management

- ☑ **Session Invalidation on Code Changes**: System tracks deployment versions to invalidate sessions when code changes
- ☑ **Version Sources**:
  - `DEPLOYMENT_VERSION` environment variable (CI/CD)
  - `NEXT_PUBLIC_BUILD_TIMESTAMP` (build-time)
  - `DEV_SERVER_START_VERSION` (development-time)
- ☑ **Middleware Validation**: Checks deployment version cookie on each request
- ☑ **Cache Rebuild**: On version mismatch, session cleared and user redirected to login for fresh permission cache

7.6 Permission-based UI: Who can see what

Project-level UI (tabs and sidebar sub-menus) is restricted by persona. The following matrix defines **who can see what** for the project workspace. This is the product source of truth for permission-based UI.

Project tabs and sidebar sub-menus (by persona):

UI element	Guest	External Collaborator	Team Member	Project Lead	Client Owner	Org Owner
Files	✅	✅	✅	✅	✅	✅
Members	❌	❌	✅	✅	✅	✅
Shares	❌	❌	✅	✅	✅	✅
Insights	❌	❌	✅	✅	✅	✅
Sources	❌	❌	✅	✅	✅	✅
Settings	❌	❌	❌	✅	✅	✅

- **Files**: Anyone who can view the project (all personas with project access).
- **Members, Shares, Insights, Sources**: Internal/collaborative tabs; visible only to Team Member, Project Lead, Client Owner, and Org Owner (not Guest or External Collaborator).
- **Settings**: Administrative; visible only to Project Lead, Client Owner, and Org Owner. Edit actions (upload, manage members, project properties) are restricted to the same roles.

View As: Org Owners can use "View as" (sidebar dropdown) to simulate another persona for testing; the UI and API respect the selected persona (via cookie) so that tab and sidebar visibility match the matrix above.

Extensibility: New tabs or personas are added via a configuration-driven permission framework (see HLD). The matrix above is implemented as capability rules (e.g. `project:can_view`, `project:can_view_internal`, `project:can_manage`) resolved from RBAC.

8. Project Members & Personas

Purpose: Manage access control through granular, role-based personas at the project level.

- [x] **File visibility**: Only users who are project members with a persona (Project Lead, Team Member, etc.) see project files. There is no special path for "org member with no project persona"; the project creator is always added as a Project Lead so they see files from creation.
- [x] **Data Model (New)**:
  - [x] **`ProjectPersona`**: Helper table scoped to `Organization`. Defines role templates.
    - **Fields**: `name`, `description`, `role` (System Role: `ORG_MEMBER` | `ORG_GUEST`), `permissions` (JSON: `can_view`, `can_edit`, `can_manage`, `can_comment`), `isDefault`.
  - [x] **`ProjectMember` Update**: Links `userId` to `projectId` AND `personaId`. Added `settings` (JSONB).
  - [x] **`ProjectInvitation`**: Tracks pending invites (`email`, `personaId`, `status`, `token`).
- [x] **Default Personas (Organization Level Templates)**:
  1. **Project Lead**
     - **System Role**: `ORG_MEMBER`
     - **Permissions**: `can_view`, `can_edit`, `can_manage`, `can_comment` (full administrative access including member management and document deletions).
     - **Access**: Continuous.
     - **Description**: Internal team member responsible for project oversight.
  2. **Team Member**
     - **System Role**: `ORG_MEMBER`
     - **Permissions**: `can_view`, `can_edit`, `can_manage`, `can_comment` (full project access including document creation, editing, and team collaboration).
     - **Access**: Continuous.
     - **Description**: Internal staff member with full project access.
  3. **External Collaborator**
     - **System Role**: `ORG_GUEST`
     - **Permissions**: `can_view`, `can_edit`, `can_comment` (can view and edit documents, provide feedback, but cannot manage project settings or delete content).
     - **Access**: Project-scoped, need-to-know basis.
     - **Description**: External partner, contractor, or vendor working on the project.
  4. **Client Contact**
     - **System Role**: `ORG_GUEST`
     - **Permissions**: `can_view`, `can_comment` (view-only access with ability to provide feedback and track progress).
     - **Access**: Stakeholder view.
     - **Description**: Client stakeholder or project sponsor with view-only access.
- [x] **UI Components (Members Tab)**:
  - [x] **Member List**: Shows User, Persona, and Status.
  - [x] **Invitation Modal**: Input Email + Select Persona.
  - [x] **Change Member Role**: Dialog to update an existing member's persona assignment.
  - [ ] **Persona Renaming** (Low Priority):

        - Allow Project Leads to rename default persona names per project (e.g., rename "Team Member" to "Accountant" or "External Collaborator" to "Contractor").
        - Simple rename field in Project > Members screen, edit button next to persona name.
        - **Scope Limitation**: Rename only - no new persona creation, no permission changes, no description editing.
    - ☑ **Invitation Status**:
        - `PENDING` : Show "Resend" button.
        - `ACCEPTED` : Link Verified / Signup in progress.
        - `JOINED` : Fully onboarded member.

- ☑ **Invitation Flow**:
    - ☑ **Sender**: Invites email -> Selects Persona -> System sends email (SMTP/Brevo).
    - ☑ **Invitee**: Clicks link -> If new, registers -> If existing, signs in -> Redirected to Project.
    - ☑ **System**: Updates `ProjectInvitation` status -> Creates `ProjectMember` record -> Assigns `OrganizationMember` role if needed.
    - ☑ **Google Drive Access**: Automatically grants folder permissions based on persona:
        - ☑ **Project Lead & Team Member**: Receive `writer` (can_edit) access to project's Google Drive folder upon joining.
        - ☑ **External Collaborator & Client Contact**: No automatic Drive access (view-only through Portal UI). Files must be explicitly assigned to these personas via File Assignment feature (see File Management).
        - ☑ **Removal**: When any member is removed from a project, their Google Drive folder access is automatically revoked. File assignments are also revoked when a member is removed.
    - ☑ **Security (Tamper-Proofing)**:
        - **Backend Enforcement**: Invitation redemption logic explicitly validates that the `invite.email` matches the `authenticated_user.email` .
        - **Prevention**: Prevents "Link Forwarding" or "UI Tampering" where a user attempts to claim an invite intended for another email address.
    - ☑ **Reliability (Transactional Integrity)**:
        - **Atomic Operations**: Acceptance logic wrapped in `prisma.$transaction` .
        - **Guarantee**: Ensures Member Creation, Org Member Creation, and Invite Status Update happen as a single atomic unit. Prevents inconsistent states (e.g. "Joined" status without "Member" record).

9. Non-Functional Requirements: Error Handling & Logging

Purpose: Ensure system reliability, observability, and robust error recovery across all application layers.

- ☑ **Infrastructure**:
    - ☑ **Sentry Integration**:
        - ☑ **Client-Side**: Browser error tracking with Session Replay (10% sampling).
        - ☑ **Server-Side**: Node.js runtime error tracking with performance monitoring.
        - ☑ **Edge Runtime**: Middleware error tracking.
        - ☑ **Source Maps**: Automated upload during build for readable stack traces.
    - ☑ **Logger Utility**:
        - ☑ **Centralized Logging**: Unified interface (`lib/logger.ts`) for all log levels (`debug` , `info` , `warn` , `error`).
        - ☑ **Environment Awareness**:
            - **Dev**: Colored console output with metadata.
            - **Prod**: JSON structured logs + Sentry integration.

        - ☑ **Context Awareness**: Captures user ID, organization context, and breadcrumbs.

- ☑ **Resilience & Recovery**:
    - ☑ **Global Error Boundaries**:
        - ☑ **Root Level**: Catches top-level crashes (`app/global-error.tsx`).
        - ☑ **Page Level**: Catches route-specific errors (`app/error.tsx`).
        - ☑ **Component Level**: Granular boundaries for widget isolation (e.g., `ProjectFileList`).
    - ☑ **Graceful Degradation**: Failed components display user-friendly fallback UI without crashing the entire app.

- ☑ **Privacy & Security**:
    - ☑ **Sanitization**:
        - ☑ Sentry masking for text input and sensitive data.
        - ☑ Removal of PII from production logs.
    - ☑ **Health Checks**: Filtered out from error tracking to reduce noise.
    - ☑ **Database Row-Level Security (RLS)**:
        - **Multi-tenancy:** At organization level (tenant = Organization). RLS applied at different levels for different tables.
        - **Org-level RLS:** For org-owned tables (`organizations` , `clients` , `projects` , `connectors` , `project_personas` , `organization_members`) restrict by `organization_id` using helper functions. **Connectors:** `portal.connectors` has organization-level RLS (policy `connectors_org_isolation`).
        - **Project-level RLS:** For project-scoped tables (`project_members` , `project_invitations`) restrict by project membership. See [HLD § Security & Compliance – RLS multi-tenancy strategy](#).
        - **Implementation:** RLS policies enforce hierarchical isolation: Organization → Client → Project → Document. Helper functions (`get_current_user_organization_ids()` , `is_user_project_member()` , etc.) enable efficient policy evaluation.

10. Waitlist System

Purpose: Build anticipation for the Pro plan launch, collect early interest, and drive viral growth through referrals.

10.1 Waitlist Signup Flow

- ☑ **Public Waitlist Page** (`/waitlist`):
    - ☑ **Fixed Email Field**: Always visible at top of page with "Enter your email" label.
    - ☑ **Email Validation**: Real-time validation checks email format and waitlist status (debounced 800ms).
    - ☑ **Email Locking**: After validation completes, email field locks with lock icon and pencil edit button.
    - ☑ **Dynamic Content Branching**:
        - ☑ **Branch 1 - Existing Waitlist Member**: Shows waitlist status (position, points, referrals), referral link, and leaderboard.
        - ☑ **Branch 2 - New Signup**: Shows registration form (company name, size, role, comments) with email pre-filled (hidden field).
        - ☑ **Branch 3 - Success**: Shows confirmation message after successful submission.
    - ☑ **Social Proof Counter**: Displays "X people have already joined!" with avatars when total count > 25.
    - ☑ **Early Access Offer Banner**: Prominently displays Pro Plus upgrade benefits and referral rewards.

- ☑ **Form Fields**:

- ☑ **Email** (required): Pre-filled from email check, hidden in form.
- ☑ **Company Name** (optional): Text input.
- ☑ **Company Size** (optional): Dropdown (Solo, 2-10, 11-50, 51-200, 200+).
- ☑ **Role** (optional): Text input (e.g., Consultant, Accountant, Founder).
- ☑ **Comments** (optional): Textarea for additional context.
- ☑ **Honeypot Field**: Hidden field to prevent bot submissions.
- ☑ **Cloudflare Turnstile**: Captcha verification for spam prevention.

- ☑ **Security & Validation**:
  - ☑ **Rate Limiting**: 3 submissions per hour per IP address.
  - ☑ **Duplicate Prevention**: Checks for existing email before submission.
  - ☑ **Server-Side Validation**: Email format, Turnstile verification, honeypot check.
  - ☑ **Privacy**: Email addresses masked in logs (first 3 chars + ***).

10.2 Referral System

- ☑ **Referral Code Generation**:
  - ☑ **Format**: 8-character alphanumeric code (uppercase, excludes confusing chars: 0/O, 1/I).
  - ☑ **Uniqueness**: Database constraint ensures unique codes.
  - ☑ **Auto-Generation**: Created automatically on waitlist signup.

- ☑ **Referral Benefits Structure**:

  For Referrers (People who share their link):

  - ☑ **Position Boost**: Move up 3 positions in waitlist per successful referral.
  - ☑ **Points System**: Earn 30 points per referral (used for leaderboard ranking).
  - ☑ **Pro Plus Upgrade**: Automatically upgraded to Pro Plus at 5 referrals.
  - ☑ **Discount**: 20% off first 3 months when subscribing.

  For Referees (People who sign up via referral link):

  - ☑ **Skip Ahead**: Start 10 positions higher than normal signups (via createdAt adjustment).
  - ☑ **Priority Access**: Early access when Pro plan launches.
  - ☑ **Discount**: 15% off first 3 months when subscribing.

- ☑ **Referral Link Sharing**:
  - ☑ **Format**: `https://pockett.io/waitlist?ref=ABC123XY`
  - ☑ **Display**: Prominently shown in waitlist status view.
  - ☑ **Copy Functionality**: One-click copy button with visual feedback.
  - ☑ **URL Parameter Handling**: Automatically detects and processes referral codes from URL.

- ☑ **Position Calculation**:
  - ☑ **Normal Signup**: Position = count of people who signed up before them (based on createdAt).
  - ☑ **Referral Signup**: Position = count before them - 10 positions (via createdAt timestamp adjustment).
  - ☑ **Referrer Boost**: For each referral, referrer's positionBoost increments by 3 (tracked in database).

10.3 Leaderboard & Social Proof

- ☑ **Leaderboard Display**:
  - ☑ **Top 10 Referrers**: Ranked by referral count (points = referrals × 30).

- ☑ **Table Format**: POSITION | EMAIL | POINTS columns.
- ☑ **Rank Badges**: Special styling for #1 (gold), #2 (silver), #3 (bronze).
- ☑ **User Highlighting**: Current user's row highlighted with purple background.
- ☑ **User Inclusion**: If user not in top 10, their row shown below with "..." separator.
- ☑ **Privacy**: Email addresses masked (first 3 chars + ***).

- ☑ **User Status Display**:
  - ☑ **Current Position**: Shows waitlist position number.
  - ☑ **Current Points**: Calculated as referrals × 30.
  - ☑ **Referral Count**: Number of successful referrals.
  - ☑ **Position Boost**: Total positions gained from referrals.
  - ☑ **Points to Move Up**: Shows how many more points needed to advance (if applicable).

- ☑ **Social Proof Counter**:
  - ☑ **Display Trigger**: Shows when total waitlist count > 25.
  - ☑ **Visual Elements**: 3 most recent joiners' avatars (initials) + "+X more" badge.
  - ☑ **Count Display**: Large, bold number with "people have already joined" text.

10.4 Database Schema

- ☑ **Waitlist Table** (`admin.waitlist`):
  - ☑ **Core Fields**: `id` , `email` , `plan` (default: "Pro"), `createdAt` .
  - ☑ **Profile Fields**: `companyName` , `companySize` , `role` , `comments` (all optional).
  - ☑ **Referral Fields**:
    - ☑ `referralCode` (String, unique, 8 chars): Unique code for sharing.
    - ☑ `referredBy` (String, nullable): referralCode of person who referred them.
    - ☑ `referralCount` (Int, default: 0): Number of successful referrals.
    - ☑ `positionBoost` (Int, default: 0): Total positions gained from referrals.
  - ☑ **Security Fields**: `ipAddress` (for rate limiting).
  - ☑ **Indexes**: `email` , `plan` , `createdAt` , `referralCode` , `referredBy` .

10.5 User Flows

- ☑ **New User Signup Flow**:
  1. User visits `/waitlist` .
  2. Enters email in fixed field at top.
  3. System validates email format and checks waitlist status (debounced).
  4. Email locks after validation.
  5. If not found: Form appears with email pre-filled (hidden).
  6. User completes profile (company, role, etc.).
  7. Completes Turnstile captcha.
  8. Submits form.
  9. Success message displayed.
  10. Email remains locked.

- ☑ **Existing User Status Flow**:
  1. User enters email in fixed field.
  2. System checks waitlist status.
  3. Email locks after validation.
  4. Status view displays:

        - Position, points, referrals.
        - Referral link with copy button.
        - Leaderboard with user's position highlighted.
        - Points needed to move up (if applicable).

- ☑ **Referral Flow**:
    1. Existing waitlist member shares referral link (`/waitlist?ref=ABC123XY`).
    2. New user clicks link.
    3. Referral code detected in URL.
    4. Banner shows "You were referred!" message.
    5. User signs up via form.
    6. System records `referredBy` field.
    7. Referee skips ahead 10 positions.
    8. Referrer's `referralCount` increments.
    9. Referrer's `positionBoost` increases by 3.
    10. Both users see updated positions.
- ☑ **Email Edit Flow**:
    1. User clicks pencil icon on locked email field.
    2. Email unlocks for editing.
    3. Status cleared (allows re-check).
    4. User edits email.
    5. System re-validates and checks status.
    6. Email locks again after validation.

10.6 Server Actions

- ☑ **`submitWaitlistForm()`** : Handles form submission with rate limiting, honeypot, Turnstile verification, duplicate checks, and referral processing.
- ☑ **`getWaitlistStatus()`** : Returns user's waitlist status (position, ahead/behind counts, referral stats).
- ☑ **`getWaitlistLeaderboard()`** : Returns top 10 referrers ranked by points, user's rank, and total count.
- ☑ **`getWaitlistCount()`** : Returns total waitlist count and recent joiners for social proof.

10.7 Admin Features

- ☑ **Internal Waitlist View** (`/internal/waitlist`):
    - ☑ **Statistics Dashboard**: Total count, breakdown by plan (Pro, Pro Plus, Business, Enterprise).
    - ☑ **Waitlist Table**: Shows all entries with email, plan, company info, referral stats, and signup date.
    - ☑ **Sorting**: Sortable by date, plan, referral count.
    - ☑ **Access**: Internal admin dashboard only.

10.8 Future Enhancements

- ☐ **Email Notifications**: Notify users when someone uses their referral link.
- ☐ **Referral Analytics**: Track referral click-through rates and conversion metrics.
- ☐ **Social Sharing**: Direct share buttons for Twitter, LinkedIn, Email.
- ☐ **Referral Milestones**: Badges and achievements for referral milestones (5, 10, 25 referrals).
- ☐ **A/B Testing**: Test different benefit structures and messaging.
- ☐ **Waitlist Management**: Admin tools to manually adjust positions, send bulk emails, export data.

---

11. Billing, Subscriptions & Payment (Polar)

Purpose: Enable paid plans (Standard, Pro, Business, Enterprise) with a single payment provider, keep organisation subscription state in sync with the provider, and support upgrade, renewal, and cancellation flows.

Payment gateway: Polar (polar.sh) is used for checkout, subscriptions, and invoicing. Polar supports global customers and provides a hosted checkout and customer portal, reducing in-app payment logic and compliance scope.

Subscription data model: Each organisation has subscription state stored in the application database (e.g. on the Organisation record or a linked Subscription record) so that feature access and UI can be decided without calling Polar on every request. Stored fields include:

- **Plan and status:** Subscription tier (e.g. Standard, Pro, Business, Enterprise), status (e.g. active, trialing, past_due, canceled, expired), and current period end date.
- **Polar linkage:** Polar customer ID and Polar subscription ID (when subscribed), so the app can reconcile webhook events and link users to the correct organisation.

Pricing and plan limits (e.g. active projects per tier) are defined in product configuration and in the [subscription planning document]; the app enforces limits and feature gates based on the stored tier and status.

Webhook integration: Polar sends subscription and order events to a dedicated webhook endpoint (e.g. `POST /api/webhooks/polar`). The app:

- **Verifies** each request using Polar's signing secret so only Polar can trigger updates.
- **Processes** events such as: subscription created/active/updated/canceled/revoked/past_due, and order created/paid, to update the organisation's subscription tier, status, and period end.
- **Handles idempotency** (e.g. by event ID) so retries do not apply the same change twice.

Outcomes for the client: customers can subscribe and manage billing via Polar; the portal always shows the correct plan and feature set; renewals and cancellations are reflected automatically after Polar sends the corresponding events.